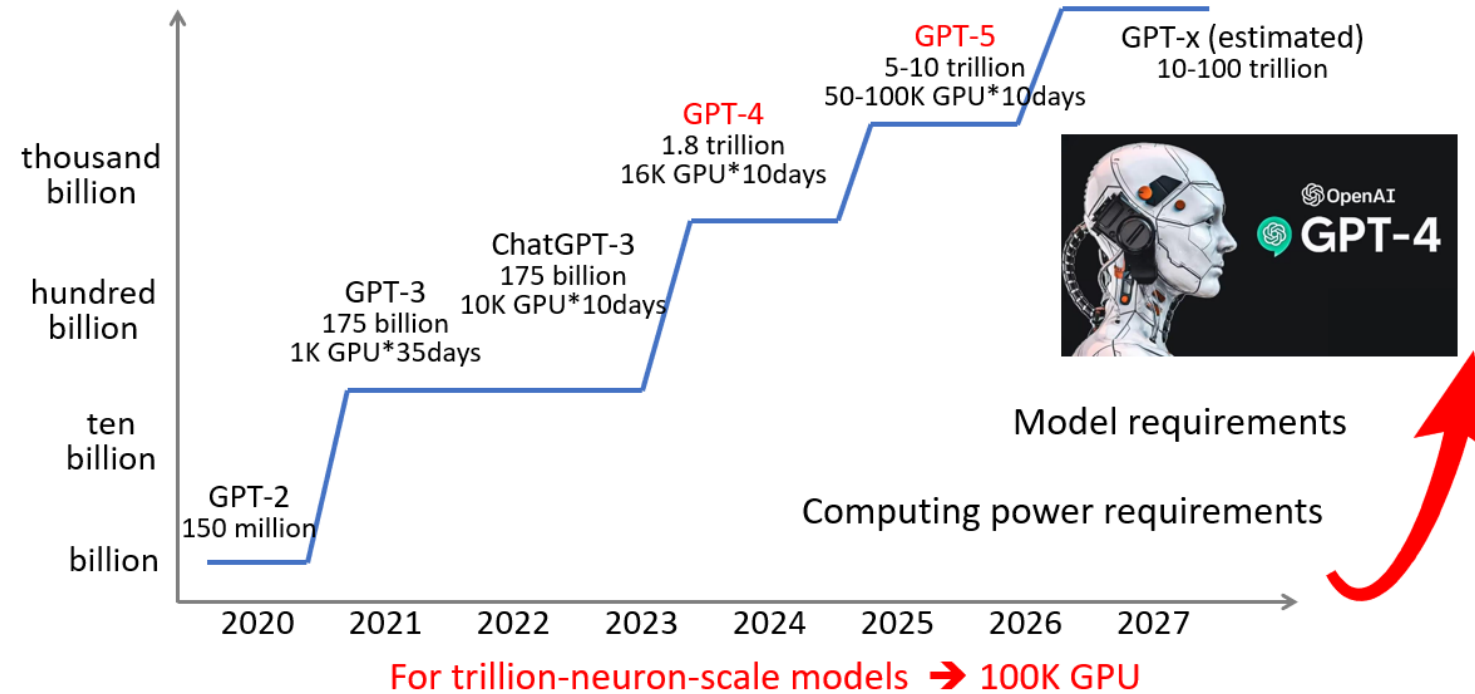


Communication Overhead of Distributed Large AI Model Training

- Deep learning models are getting larger
 - Distributed across various nodes/servers, each with multiple GPUs
- Incurs network communication overhead
 - GPUs can spend as much as **20% - 65% of time idle waiting** on network communication [1]



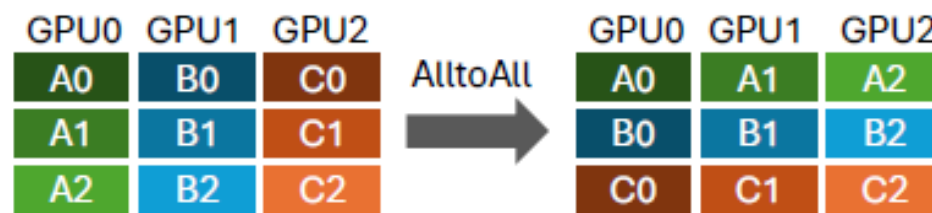
[1] Sapio, Amedeo, et al. "Scaling distributed machine learning with in-network aggregation." NSDI'21

Communication in Distributed ML

- **Collective communication**: abstractions for communication to exchange intermediate results (e.g., gradients, activations) for distributed ML
 - E.g., AllGather, AlltoAll, AllReduce



(a) Illustration of AllGather



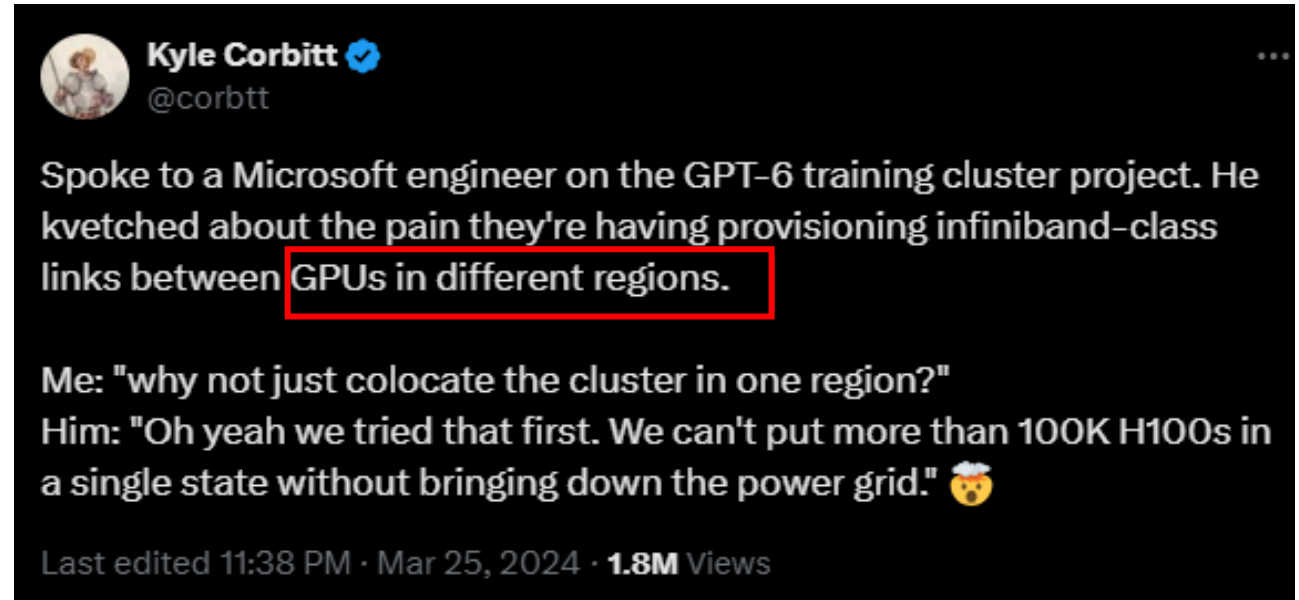
(b) Illustration of AlltoAll

- Collective communication request: demand for collective communication operations
- **Collective communication libraries (CCL)**: define routing and scheduling decision to exchange data based on network status

[1] <https://docs.nvidia.com/deeplearning/nccl/user-guide/docs/usage/collectives.html>

Motivation for Inter-DC CCL

- With the increasing of model size, it is a non-trivial to deploy required GPUs in the same DC
 - Space, power, power density, and cooling requirement
- Previous works on CCL mainly focus on intra-DC network

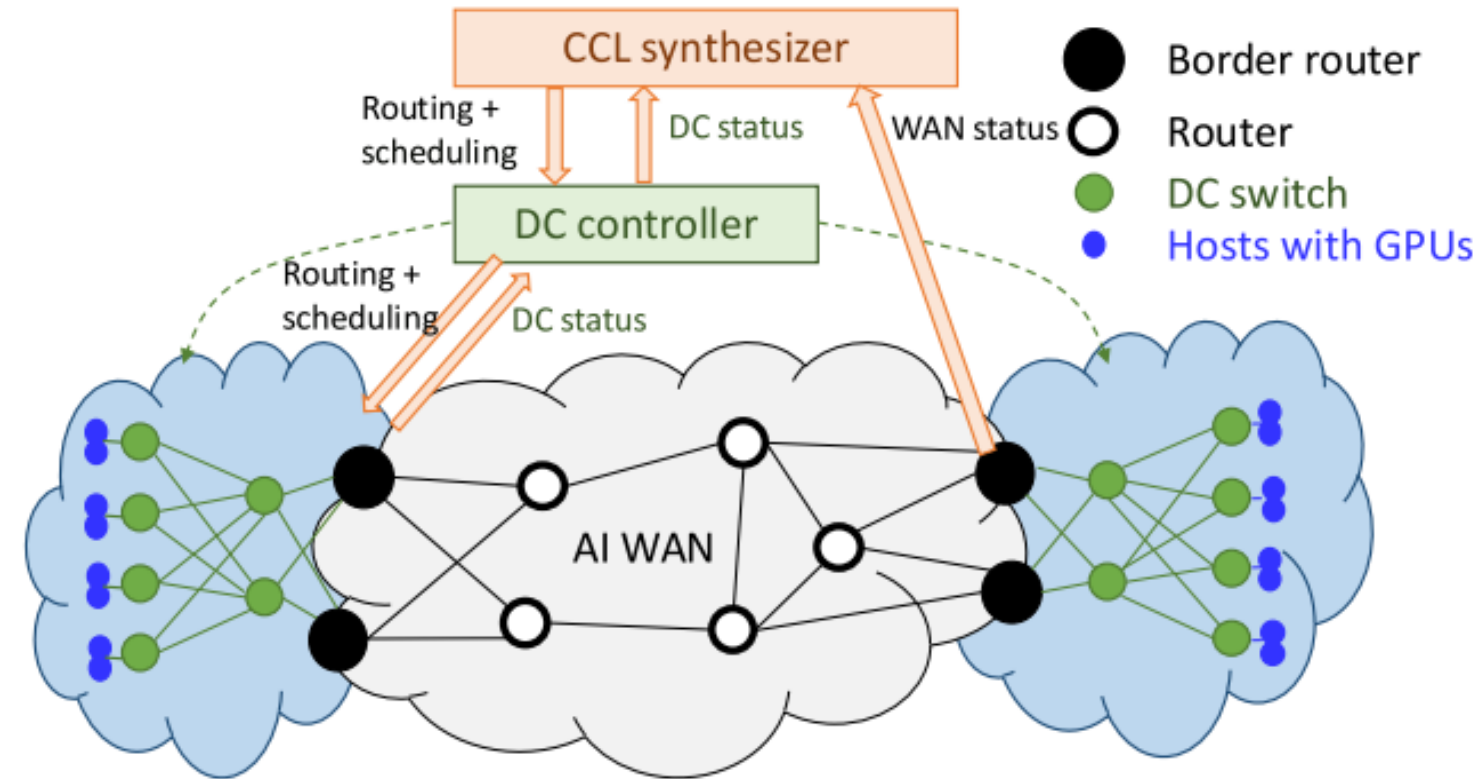


A CCL tailored for inter-DC networks is required!

[1] Gandhi, Rohan, Karan Tandon, Debopam Bhattacharjee, and Venkata N. Padmanabhan. "Improving training time and GPU utilization in geo-distributed language model training." *arXiv preprint arXiv:2411.14458* (2024).

[2] Liu, Xuting, Behnaz Arzani, Siva Kesava Reddy Kakarla, Liangyu Zhao, Vincent Liu, Miguel Castro, Srikanth Kandula, and Luke Marshall. "Rethinking machine learning collective communication as a multi-commodity flow problem." *In Proceedings of the ACM SIGCOMM 2024 Conference*, pp. 16-37. 2024.

Challenges in CCL Optimization over Multiple DCs



WAN network:

- **Variable background traffic** and routing shift
- Asymmetric, with forward and reverse **paths differing in delay and capacity**

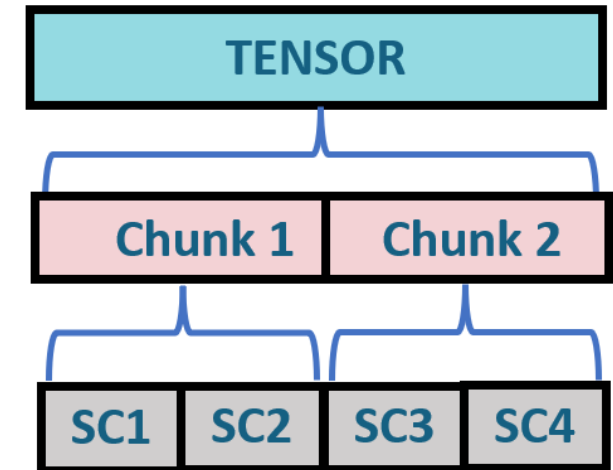
- Most CCLs assume uniform, high-bandwidth environments with static topology routing
- **Bottleneck: collective communication across WANs**

Outline

- Introduction of Coca
 - Background and Motivation
 - **System Model**
- Fast Pace: Simulation Design and Setting for Fast Pace
 - Design of Agent
 - Benchmarking SimAI and Customized Simulator
 - Problem Statement of CCL in Multi-Tenant GPU Clusters
- Progress on Testbed
 - Scope & Plan
 - Evaluating the Impact of Customized CCL on Collective Completion Time
 - Evaluating the Impact of Customized CCL on Model Training Time
- Next Step

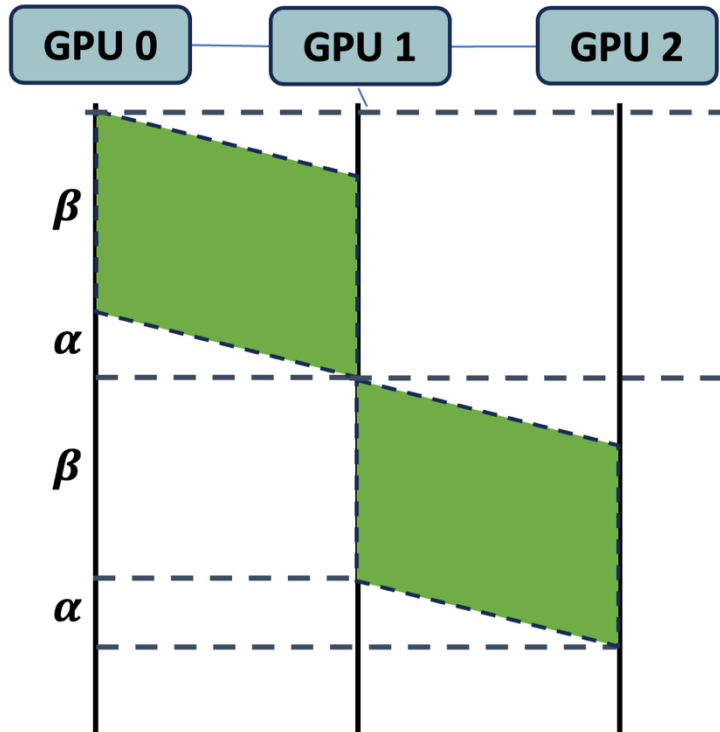
System Model

- Collective communication type: All Gather, All to All, All Reduce, etc.
- **Tensor**: multi-dimensional array (e.g., matrix) commonly used to represent model parameters in deep learning
 - Tensor is **partitioned** into multiple **chunks**
 - **Chunk size**: data size of the chunk, which is the amount of data to be transmitted per GPU in one collective communication
 - **Subchunk (SC) size**: concept introduced by us to investigate impacts of chunk size



System Model

Transmission delay and propagation delay

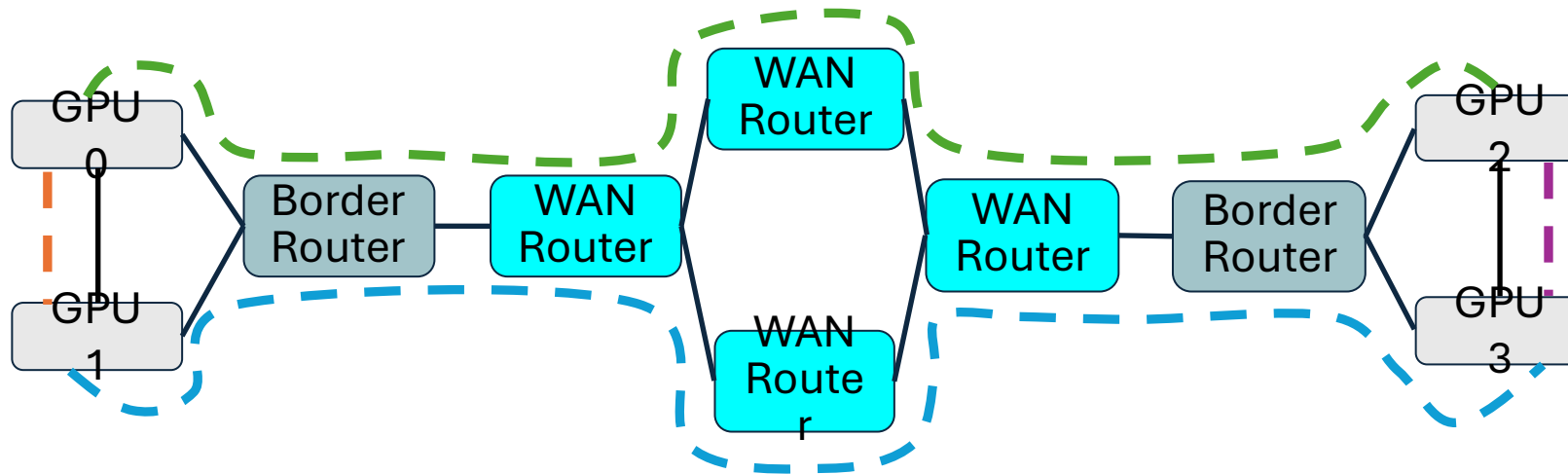


- **Propagation delay (α):**
 - Time taken for a signal to travel from the sender to the receiver through the physical medium
 - Calculated as: distance divided by propagation speed in the medium
- **Transmission delay (β):**
 - Time required to push all the bits of a packet into the transmission medium
 - Calculated as: data size divided by the transmission rate of the link
- **Transfer delay: transmission delay + propagation delay**

System Model

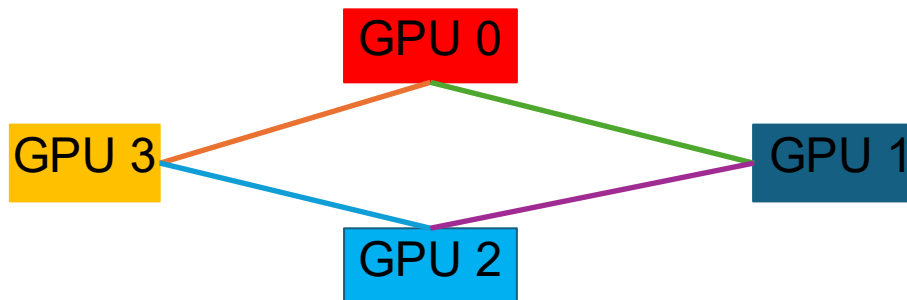
Physical topology and logical topology

- Heterogeneous physical topology



Physical path of the virtual link

- Logical topology (e.g., NCCL) :



Example: Routing and Scheduling of CCL on Logical Topology

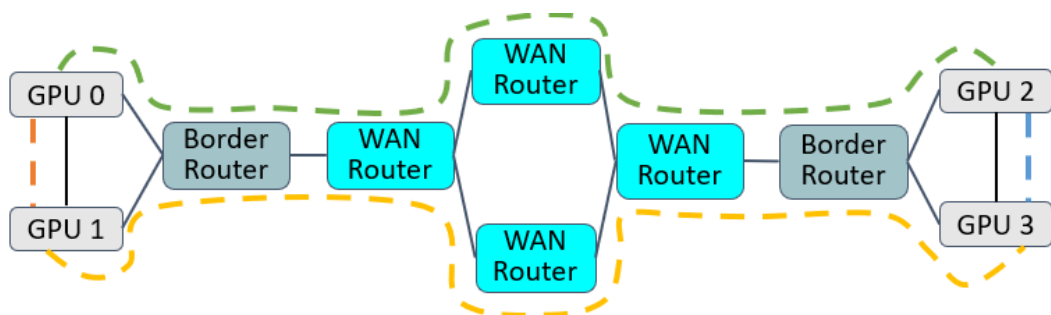


Fig. 1 Physical topology abstraction

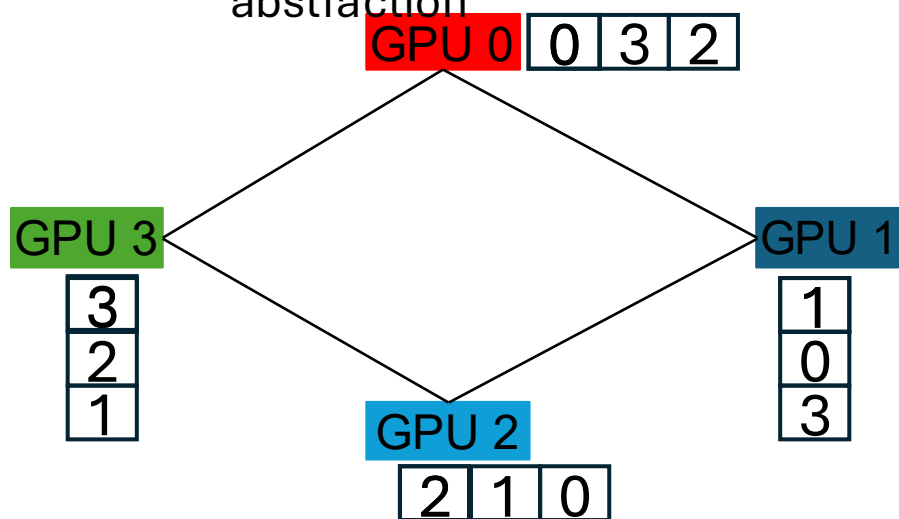


Fig. 2 Example of Routing and Scheduling in logical topology for AllGather

Epoch: a short time period for transmitting chunks (e.g., shortest transfer delay)

- Epoch 1:
 - Chunk0: GPU0 -> GPU1
 - Chunk1: GPU1 -> GPU2
 - Chunk2: GPU2 -> GPU3
 - Chunk3: GPU3 -> GPU0
 - Epoch 2:
 - Chunk0: GPU1 -> GPU2
 - Chunk1: GPU2 -> GPU3
 - Chunk2: GPU3 -> GPU0
 - Chunk3: GPU0 -> GPU1
 - Epoch 3:
 - Chunk0: GPU2 -> GPU3
 - Chunk1: GPU3 -> GPU0
 - Chunk2: GPU0 -> GPU1
 - Chunk3: GPU1 -> GPU2
- One epoch
- One **iteration/round** (time to fulfill one collective communication demand)

Teacher–Student-Model-Based Transfer CCL Optimization for Variable Network Parameters



POLITECNICO
MILANO 1863

Collective Communication Library (CCL): *Library use for synchronizing data across GPUs in distributed training^[1]*

Motivation:

1. For large topologies (e.g., thousands of GPUs), even heuristic solutions take hundreds of seconds
2. ILP-based methods cannot adapt fast enough when network parameters change
3. We embed a neural **CCL Synthesizer into the Idea 1 framework** to generate near-optimal scheduling instantly under dynamic network conditions

Key Idea:

We introduce a **Teacher–Student transfer framework**:

- **Teacher (TECCL^[1] / SCALE-CCL^[2])** provides optimal or high-quality solutions
- **Student (Neural Agent)** learns to approximate and generalize them across network scales

Component	Role	Function
 Agent (Decision NN)	Student	Generates the collective communication policy (scheduling decision)
 SimNet (Simulator NN)	Teacher	Approximates the real simulator and estimates the collective completion time for each policy

[1] Liu X, Arzani B, Kakarla S K R, et al. Rethinking machine learning collective communication as a multi-commodity flow problem[C]//Proceedings of the ACM SIGCOMM 2024 Conference. 2024: 16-37.

[2] Xiong J, Zhang Q, Medagliani P, et al. SCALE-CCL: A Scalable Collective Communication Library for Wide-Area Distributed Training[C]//Proceedings of the 1st Workshop on Inter-networking challenges for AI. 2025: 21-27.

Idea 2: Teacher–Student-Model-Based Transfer Optimization for Variable Network Parameters

Step 1. Agent Pre-training

- Supervision = ILP-generated optimal schedules
- Agent learns to generate feasible / near-optimal solutions under different topologies
- Save intermediate solutions (high / medium / low quality) for SimNet

Agent

GNN/Graph Transformer

Step 2. SimNet Pre-training

- Input: (topology, schedule) → Output: predicted collective time
- Training data = {ILP optimum + Agent intermediate solutions}
- Learns the full distribution of solution quality → not just one optimal point

SimNet

GNN/Graph Transformer

Idea 2: Teacher–Student-Model-Based Transfer Optimization for Variable Network Parameters



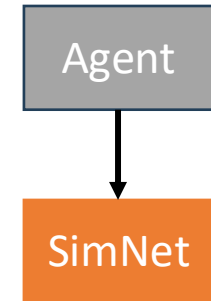
POLITECNICO
MILANO 1863

Step 3. SimNet Calibration

- Freeze **Agent**, feed its output (scheduling decisions) to **SimNet**
- Compare **SimNet-predicted time** with the **real simulator (ground-truth)**
- If the gap $>$ threshold τ (e.g., 5%), locally retrain SimNet for calibration



Goal: Align SimNet with the real simulator within Agent's explored region



Step 4. Agent Fine-tuning (SimNet frozen) — compact

- Freeze **SimNet**; update **Agent** only
- Agent $\rightarrow$ SimNet $\rightarrow$ Predicted reward $\rightarrow$ Agent update.
- Occasionally validate with the real simulator (GT)



Goal: Agent learns faster with SimNet as a differentiable teacher, while maintaining fidelity to real simulator

